HCLSoftware

Deploy HCL DX Compose in Minutes

November 12, 2026



HCLSoftware

DX Webinar

An ongoing series of product-focused, hands-on sessions highlighting features and functionality, with an emphasis on demos and visuals.

Watch for your invite to our next session!



HCL **Digital Experience**

HCLSoftware

Webinar

Deploy HCL DX Compose in Minutes:

Live Demo & Expert Insights!

November 12, 2026 | 5 PM CET



Andrés Gorostidi Pulgar
EuropeSIP DX Expert



Herbert Hilhorst
HCLSoftware TX
Customer Success Engineer

Presentation

Deploy HCL DX Compose in Minutes

Ready to take the guesswork out of deploying DX Compose on Kubernetes?

Hands-on session with our trusted Business Partner, EuropeSIP, where we strip away the complexity and build a deployment live—step by step, iteration by iteration.

If you already hold DX licenses, you can deploy DX Compose across your non-production environments today at no extra licensing hurdle!

Deploy DX Compose in Action

Scenario: Harry Pappus, a DX admin, deploys DX Compose, through a practical, step-by-step incremental approach—building real capabilities one iteration at a time:

1. Basic deployment with embedded Derby
2. LDAP via Kubernetes Secrets
3. External database
4. AI assistant (IQ)
5. Search integration



Harry Pappus

DX Admin, Nex Haven

Our starting point: the cluster

Kubernetes

- **k3s v1.35.5**, CNCF-certified Kubernetes, managed with Rancher
- **3 control-plane + 3 worker** nodes, Ubuntu 24.04, x86-64
- Workstation: **kubectl** and **Helm 4.3**
- Access to the **HCL Harbor** (hclcr.io) with our CLI secret

DNS, ingress, storage

- **DNS** at our provider: *.apps.promox.europesip-lab.com already points to the cluster
- **Ingress:** ingress-nginx behind MetalLB
- **StorageClasses** (Longhorn 1.12): **dx-deploy-rwo** (ReadWriteOnce) and **dx-deploy-rwx** (ReadWriteMany, for DAM)

Minimum free resources

- **Base DX:** ~5 CPU, ~11 GiB
- **+ LDAP and PostgreSQL:** +0.5 CPU, +1 GiB
- **+ IQ and LiteLLM:** +0.5 CPU, +1.5 GiB
- Start-up peak with limits: ~8 CPU, ~14 GiB, spread over the nodes

Checked before the session, read-only: **common/00-check-prereqs.sh** (StorageClasses, IngressClass, DNS, Harbor credentials).

Pre-installed: the external dependencies

OpenLDAP

- Demo users (andres, macarena…) and groups (**dxadmins**, dxauthors)
- openldap.dx-shared:389

PostgreSQL 18

- CloudNativePG operator
- **dx_core** prepared for the Web Engine, **iqdb** for the IQ sessions
- dx-pg-rw.dx-shared:5432

LiteLLM

- Proxy between IQ and the AI models
- OpenRouter → **Claude Sonnet 4.6** / Haiku 4.5
- litellm.dx-shared:4000

Third-party software we **assume is already installed**: we do not cover its installation today, for lack of time. The labs (link at the end) have the instructions and scripts: the process is **automated** and takes **no more than 3 minutes**. · **dx-demo** = HCL software only (no DX yet: we build it live) · **dx-shared** = these third-party tools.

AGENDA

Today: a production-style deployment

Lab	What we build	⏱ Run time
1	**Base DX** DX Compose CF239 on Derby, published	**~6 min***
2A	**LDAP** DX users and groups from OpenLDAP	**~5 min**
2B	**External database** Web Engine data: Derby → PostgreSQL	**~13 min**
3A	**AI assistant** IQ + LiteLLM, sessions in PostgreSQL	**~8 min**
4	**Search** OpenSearch and Search V2	**~7 min**
Total run time		**~39 min**

How we run it

- A ready-made folder, **dx-install/**, generated for this cluster with every value filled in
- **RUNBOOK.md**: every step with its command (install.sh runs the same ones)
- We run it step by step; while Helm works, we explain the next lab
- Production practice: **values files + secrets + Helm**, external LDAP and database
- * The nodes already have the images: **15-25 min** on a new cluster

Base DX: DX Compose on Derby

⏱️ ~6 min*

What we do

- Namespace **dx-demo**, pull secret for hclcr.io, **wpsadmin** and a TLS certificate
- Chart **hcl-dx-deployment 2.50.0** (CF239): Web Engine, HAProxy, DAM, Persistence Node
- Publish it: an Ingress that re-encrypts towards HAProxy
- Result: the portal on **Derby**, basic user registry

Commands · RUNBOOK steps 1-2 to 1-6

```
helm registry login hclcr.io -u <user> -p <cli-secret>
helm pull oci://hclcr.io/dx-compose/hcl-dx-deployment \
     --version 2.50.0 -d charts
kubectl apply -f 1-dx/secrets.yaml
kubectl -n dx-demo create secret tls dx-tls-cert \
     --cert=1-dx/dx-tls.crt --key=1-dx/dx-tls.key

helm upgrade --install dx-deployment \
     charts/hcl-dx-deployment-2.50.0.tgz -n dx-demo \
     -f 1-dx/custom-values.yaml --timeout 30m --wait=legacy
kubectl apply -f 1-dx/ingress-dx.yaml

# in another terminal
kubectl -n dx-demo get pods -w
```

Key values

- applications: **webEngine** true, core false, openLdap false
- networking.webEngine.host: **dx-demo.apps.promox.europesip-lab.com**
- **dx-deploy-rwo**: Web Engine, Persistence Node, License Manager
- **dx-deploy-rwx**: DAM binaries
- images: hclcr.io + **regcred**

▶️ **Live demo:** walk through the significant values in **1-dx/custom-values.yaml**, then helm install, watch the pods start and log in to /wps/portal as wpsadmin.

LDAP: DX users from OpenLDAP

⏱ ~5 min

What we do

- DX authenticates against the **pre-installed OpenLDAP**
- Native **LDAP block** of the chart; bind credentials in a Kubernetes **Secret**
- One **helm upgrade**: the Web Engine restarts with the LDAP registry
- Then: **Administrator** role for the dxadmins group

Commands · RUNBOOK step 2A-3

```
# the bind secret dx-ldap-bind comes from 2a-ldap/secrets.yaml

helm upgrade --install dx-deployment \
     charts/hcl-dx-deployment-2.50.0.tgz -n dx-demo \
     -f 1-dx/custom-values.yaml \
     -f 2a-ldap/custom-values.ldap.yaml \
     --timeout 30m --wait=legacy

# every upgrade carries ALL the layers so far:
# one more -f per lab, never --reuse-values
```

Key values

- ldap.type: **other** · port **389**
- host: **openldap.dx-shared.svc.cluster.local**
- suffix (base DN): **dc=dxdemo,dc=com**
- customLdapSecret: **dx-ldap-bind**
- groups: groupOfUniqueNames / uniqueMember

▶️ **Live demo:** walk through the significant values in **2a-ldap/custom-values.ldap.yaml**, then the upgrade; log in as macarena (LDAP) and give dxadmins the Administrator role.

External database: Derby → PostgreSQL

⏱ ~13 min

What we do

- The Web Engine moves its data from Derby to the **pre-installed PostgreSQL 18**
- Two secrets with the **dbtype / dbdomain** properties
- One **helm upgrade** with useExternalDatabase: the chart **transfers** the data
- PostgreSQL needs the table script from HCL (not validated yet); else Derby or Oracle

Commands · RUNBOOK steps 2B-4 and 2B-5

```
kubectl -n dx-demo create secret generic \
     custom-credentials-webengine-dbtype-secret \
     --from-file=2b-postgres/dx_dbtype.properties
kubectl -n dx-demo create secret generic \
     custom-credentials-webengine-dbdomain-secret \
     --from-file=2b-postgres/dx_dbdomain.properties

helm upgrade --install dx-deployment ... \
     -f 1-dx/custom-values.yaml -f 2a-ldap/custom-values.ldap.yaml \
     -f 2b-postgres/custom-values.postgres.yaml

kubectl -n dx-demo logs -f dx-deployment-web-engine-0 \
     -c system-out-log
```

Key values

- useExternalDatabase: **true** · dropDatabaseTables: false
- JDBC: **jdbc:postgresql://dx-pg-rw.dx-shared.svc.cluster.local:5432/dx_core**
- user **dxuser**, one schema per domain
- Result: **465 tables** transferred

▶️ **Live demo:** walk through the significant values in **2b-postgres/custom-values.postgres.yaml** and **dx_dbdomain.properties**, follow the transfer log, count the tables.

AI assistant: HCL DX IQ

⏱️ ~8 min

What we do

- IQ = **Integrator + MCP Server** (chart hcl-dx-iq 1.0.3), next to DX
- LLM through the **pre-installed LiteLLM** → OpenRouter (Claude), instead of AWS Bedrock
- Chat sessions in **PostgreSQL** (iqdb) instead of memory
- One DX **helm upgrade**: HAProxy routes the IQ chat

Commands · RUNBOOK steps A1, 3-4 and 3-5

```
kubectl -n dx-shared exec dx-pg-1 -c postgres -- psql -U postgres \
     -c "CREATE DATABASE iqdb OWNER dx_iq_db_user"

helm upgrade --install dx-iq charts/hcl-dx-iq-1.0.3.tgz \
     -n dx-demo -f 3-iq/custom-iq-values.yaml \
     -f 3-iq/custom-iq-values.db-external.yaml

helm upgrade --install dx-deployment ... \
     -f 1-dx/custom-values.yaml -f 2a-ldap/custom-values.ldap.yaml \
     -f 2b-postgres/custom-values.postgres.yaml \
     -f 3-iq/custom-values.iq.yaml
```

Key values

- LiteLLM: **http://litellm.dx-shared.svc.cluster.local:4000** (liteLlmUrl + LITELLM_BASE_URL)
- iq-general-purpose → **Claude Sonnet 4.6**; iq-summary → Haiku 4.5
- MCP: enableWcm, enableDam
- networking.dxIqService: **dx-iq-integrator**

▶️ **Live demo:** walk through the significant values in **3-iq/custom-iq-values.yaml**, then the IQ ✨ button in the DX toolbar: “Which WCM libraries are there?”

Search with OpenSearch

⏱️ ~7 min

What we do

- **Search V2**: OpenSearch + Search Middleware + File Processor (hcl-dx-search 2.37.0)
- Mutual TLS: a lab CA and **admin / node / client** certificates as secrets
- One DX **helm upgrade** enables Search V2

Commands · RUNBOOK steps 4-1 to 4-4

```
# 4-1: lab CA + admin, node and client certificates
#      -> secrets search-admin-cert, search-node-cert, ...

helm upgrade --install dx-search \
     charts/hcl-dx-search-2.37.0.tgz -n dx-demo \
     -f 4-search/custom-search-values.yaml --timeout 20m

helm upgrade --install dx-deployment ... \
     -f ... -f 3-iq/custom-values.iq.yaml \
     -f 4-search/custom-values.search.yaml

curl -sk https://dx-demo.apps.promox.europesip-lab.com\
/dx/api/search/v2/explorer/
```

Key values

- adminDN: **CN=admin,OU=DX Compose Labs,O=EUROPESIP,C=ES**
- Crawlers: **WCM, JCR, portal and DAM**, every 15 min
- searchMiddlewareService: **dx-search-search-middleware-query**
- dx-deploy-rwo for the OpenSearch data

▶️ **Live demo:** walk through the significant values in **4-search/custom-search-values.yaml**, then the Search Center and the search API explorer.

Solution

Deploy DX Compose in Minutes

You can deploy HCL DX Compose, connect to LDAP, transfer to postgreSQL, enable search, IQ and more in no time

1. **Replay this webinar** to review the steps ‘live’ again.
2. **Use the GitHub** with documentation details on each step.
3. **Get our support** - our Customer Support Engineers are ready to further support you.

HCLSoftware

Questions?

You can still add any questions to the Q&A panel, if you haven’t already.

We will address any queries that we don’t get to today in our follow-ups.



References: the labs and their tools

The labs

Guides and scripts of every iteration

github.com/europesip/hcl-dx-compose-labs

Configuration wizard

Generates env.sh, a RUNBOOK or the dx-install/ folder used today

europesip.github.io/hcl-dx-compose-labs

Disclaimer: the labs also include **optional** examples and additional labs, such as the **automated deployment of Script Applications with a Helm chart** or **extending HCL DX IQ with a custom MCP Server**. We did not show them today for lack of time and to keep the focus on a typical installation. They are provided **as-is**: fully functional, but **not officially supported**, as they are material still in development.

Thank you!

Deploy HCL DX Compose in Minutes

github.com/europesip/hcl-dx-compose-labs
europesip.github.io/hcl-dx-compose-labs